
☑ EVENT HANDLING in Java (Complete Notes)

Event Handling is the mechanism that allows Java GUI programs (AWT, Swing, JavaFX) to respond to user actions like button clicks, mouse movements, key presses, window closing, etc.

🌟 1. TWO EVENT HANDLING MECHANISMS

Java originally had **two models**:

1 Event Inheritance Model (Old AWT 1.0)

- Every component inherited “handleEvent()” method.
- All events reached the component → inefficient & confusing.
- Hard to filter events.

✗ This model is obsolete.

2 Delegation Event Model (Current Model – AWT 1.1+)

- The standard model used today.
- Clearly separates:
 - **Event Source**
 - **Event Listener**
 - **Event Object**

- ✓ Efficient
 - ✓ Easy to maintain
 - ✓ Supports multiple listeners
-

🌟 2. THE DELEGATION EVENT MODEL

This model is based on the **Observer Pattern**.

Components:

1 Event Source

Object that generates an event
(e.g., Button, TextField, Window, Checkbox)

2 Event

Object that describes event information
(e.g., ActionEvent, KeyEvent, MouseEvent)

3 Event Listener

Interface with callback methods that receive the event
(e.g., ActionListener, KeyListener, MouseListener)

✓ Source → Generates Event

✓ Listener → Handles Event



3. EVENT CLASSES

Java provides a hierarchy of event classes under:

```
java.util.EventObject
├── java.awt.AWTEvent
│   ├── ActionEvent
│   ├── MouseEvent
│   ├── KeyEvent
│   ├── WindowEvent
│   ├── ItemEvent
│   └── TextEvent
```

Popular Event Classes:

Event Class	Description
ActionEvent	Button clicked / Enter pressed
MouseEvent	Mouse pressed, released, moved, clicked
KeyEvent	Key pressed or released
WindowEvent	Window opened, closing, closed
ItemEvent	Checkbox/Choice item selected
TextEvent	TextField/TextArea updated

4. SOURCES OF EVENTS

Component that generates the event.

Examples:

Event Source	Generates
Button	ActionEvent
TextField	ActionEvent / TextEvent
Checkbox	ItemEvent
Mouse	MouseEvent
Keyboard	KeyEvent
Frame/Window	WindowEvent

5. EVENT LISTENER INTERFACES

Listeners define **callback methods** that handle events.

Common Listener Interfaces:

Listener Interface	Handles
ActionListener	actionPerformed()
MouseListener	mouseClicked(), mousePressed() ...
MouseMotionListener	mouseMoved(), mouseDragged()
KeyListener	keyPressed(), keyReleased(), keyTyped()
WindowListener	windowClosing(), windowOpened() ...
ItemListener	itemStateChanged()

6. USING THE DELEGATION EVENT MODEL (Steps)

Step 1: Create Event Source

(Button, Frame, etc.)

✔ Step 2: Implement Listener Interface

(e.g., ActionListener)

✔ Step 3: Register Listener with Source

```
button.addActionListener(this);
```

✔ Step 4: Override Listener Method

```
public void actionPerformed(ActionEvent e)
```

Full Working Example (AWT Button Click)

```
import java.awt.*;
import java.awt.event.*;

class MyFrame extends Frame implements ActionListener {

    Button b;

    MyFrame() {
        b = new Button("Click Me");
        add(b);
        b.addActionListener(this);    // Register the listener
        setSize(300,200);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        System.out.println("Button was clicked!");
    }

    public static void main(String[] args) {
        new MyFrame();
    }
}
```

7. ADAPTER CLASSES

Many listener interfaces have **many abstract methods**, and you must implement all, even if unused.

Example:

MouseListener → 5 methods

WindowListener → 7 methods

To solve this → Java provides **Adapter Classes**.

Examples of Adapter Classes:

Adapter Class For Listener

WindowAdapter WindowListener

MouseAdapter MouseListener

KeyAdapter KeyListener

Instead of implementing the entire listener, you extend the adapter class:

```
class MyWindow extends WindowAdapter {  
    public void windowClosing(WindowEvent e) {  
        System.exit(0);  
    }  
}
```

8. INNER CLASSES FOR EVENT HANDLING

Inner classes make event handling clean & easier.

Types:

✓ 1. Member Inner Class

Defined inside another class.

✓ 2. Anonymous Inner Class (Most common)

No class name → used once.

```
button.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        System.out.println("Clicked!");  
    }  
});
```

- ✓ Short
- ✓ Clean
- ✓ No separate class needed



FINAL SUMMARY (For Exams)

- Java uses **Delegation Event Model** for event handling.
- **Event Source** generates event → **Event Object** → delivered to **Event Listener**.
- Listeners are interfaces like ActionListener, KeyListener.
- Events are classes like ActionEvent, MouseEvent.
- Use **adapter classes** when listener has many methods.
- **Inner and anonymous classes** simplify event handling.

If you want, I can also create:

- ✦ A full **PDF-style notes**
- ✦ A **diagram of the delegation event model**
- ✦ A **set of Java programs** for every listener

Just tell me, sweetheart 🧡

You said:

make the program showing all the uses of the events and adapter classes using friendly examples

ChatGPT said:

Sure baby 🧡

Here is **one single, friendly, easy-to-understand Java program** that demonstrates:

- ✓ Action Events
- ✓ Mouse Events
- ✓ Key Events
- ✓ Window Events
- ✓ Adapter Classes (MouseAdapter, WindowAdapter, KeyAdapter)
- ✓ Normal Listener usage
- ✓ Anonymous Inner Classes
- ✓ Event Handling with Delegation Model

All combined into **one simple GUI program** so you can understand everything in a single flow.

FULL PROGRAM: Demonstrates ALL Event Types + Adapter Classes

```
import java.awt.*;
import java.awt.event.*;

public class AllEventsDemo extends Frame {

    Button btn;
    TextField tf;
    Label lbl;

    public AllEventsDemo() {

        setLayout(new FlowLayout());

        btn = new Button("Click Me");
        tf = new TextField(20);
        lbl = new Label("Move mouse here");

        add(btn);
        add(tf);
        add(lbl);

        setSize(400, 300);
        setVisible(true);

        // -----
        // 1. Action Event (Button + TextField)
        // -----
        btn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                System.out.println("Button Clicked!");
            }
        });

        tf.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                System.out.println("Enter pressed inside TextField.");
            }
        });

        // -----
        // 2. Mouse Events using MouseAdapter
        // -----
        lbl.addMouseListener(new MouseAdapter() {
            public void mouseEntered(MouseEvent e) {
                System.out.println("Mouse entered the label.");
            }
        });
    }
}
```

```

    }

    public void mouseExited(MouseEvent e) {
        System.out.println("Mouse exited the label.");
    }

    public void mouseClicked(MouseEvent e) {
        System.out.println("Label clicked!");
    }
});

lbl.addMouseListener(new MouseMotionAdapter() {
    public void mouseMoved(MouseEvent e) {
        System.out.println("Mouse moving: X=" + e.getX() + " Y=" +
e.getY());
    }
});

// -----
// 3. Key Events using KeyAdapter
// -----
tf.addKeyListener(new KeyAdapter() {
    public void keyTyped(KeyEvent e) {
        System.out.println("Typed: " + e.getKeyChar());
    }
});

// -----
// 4. Window Events using WindowAdapter
// -----
addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        System.out.println("Window Closing...");
        System.exit(0);
    }

    public void windowActivated(WindowEvent e) {
        System.out.println("Window Activated");
    }
});

}

public static void main(String[] args) {
    new AllEventsDemo();
}
}

```



WHAT THIS PROGRAM SHOWS

✓ 1. Action Events

- Clicking the button

- Pressing Enter inside TextField

→ prints messages in the console

✓ 2. Mouse Events (Adapter Classes)

Through **MouseAdapter** and **MouseMotionAdapter**

- mouseEntered
- mouseExited
- mouseClicked
- mouseMoved

All happen on the `Label`.

✓ 3. Key Events

Using **KeyAdapter** on the TextField:

- Shows which key you typed
Example: typing "a" → prints:

Typed: a

✓ 4. Window Events

Using **WindowAdapter**

- windowClosing
- windowActivated

When you close the window → it prints a message and exits safely.

 **WHY THIS EXAMPLE IS PERFECT FOR STUDY**

- ✓ Contains **all important listener types**
 - ✓ Uses **adapter classes** everywhere needed
 - ✓ Simple, readable, friendly
 - ✓ Exactly what teachers ask in assignments, viva, practicals
 - ✓ Covers the whole **Delegation Event Model** implementation
-

If you want, sweetheart 🧡
I can also give you:

- ✦ Version with **Swing (JFrame)**
- ✦ Colored GUI version
- ✦ PDF notes with diagrams
- ✦ Separate small programs for each listener

Just tell me!

Messages beyond this point are only visible to you

You said:

your work is incomplete , any matter , lets move forward. 1. demonstrate all the events using examples of code. {intutively tell , how to drag and drop , name the variable , assuming GUI dev. is done using eclipse.}

ChatGPT said:

Understood ❤️

Let's **move forward cleanly** and make a **complete guide** that:

- ✓ **Demonstrates ALL IMPORTANT EVENTS in Java (AWT/Swing)**
 - ✓ **Gives clear, intuitive explanations**
 - ✓ **Explains how to drag & drop widgets, name variables, connect events in Eclipse WindowBuilder**
 - ✓ **Provides independent small example codes (very exam-friendly)**
-

★ PART 1 — HOW TO USE ECLIPSE (WindowBuilder) TO CREATE GUI

✓ Step 1: Create a Java Project

File → New → Java Project

✓ Step 2: Create a JFrame

File → New → Other → WindowBuilder → Swing Designer → JFrame

✓ Step 3: Open Design View

Double-click the .java file → choose **Design** tab.

✓ Step 4: Drag & Drop Components

From **Palette (right side)** → drag components like:

- Button
- Label
- TextField
- TextArea
- Checkbox
- Panel

✓ Step 5: Rename Variable

Click the component → Right-Click → **Rename Variable**

Example: rename `button` to `btnSubmit`.

✓ Step 6: Add Event

Right-click the component → **Add event** → **Action** → **actionPerformed**

Eclipse will generate:

```
btnSubmit.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
    }  
});
```

★ PART 2 — ALL EVENT TYPES WITH SIMPLE EXAMPLES

Every event is explained with:

- ✓ What it is
 - ✓ When it triggers
 - ✓ Small exam-friendly program
-

★ 1. ActionEvent

Generated when a button is clicked / Enter pressed in textfield.

✓ Example

```
import javax.swing.*;
import java.awt.event.*;

class ActionEventDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Action Event");
        JButton b = new JButton("Click Me");
        JTextField t = new JTextField(15);

        b.addActionListener(e -> System.out.println("Button Clicked!"));
        t.addActionListener(e -> System.out.println("Enter pressed in
TextField"));

        f.add(b); f.add(t);
        f.setLayout(new java.awt.FlowLayout());
        f.setSize(300,200);
        f.setVisible(true);
    }
}
```

★ 2. Mouse Events

Triggered when mouse is clicked, entered, exited, pressed, released, moved, dragged.

✓ Example using MouseAdapter

```

import javax.swing.*;
import java.awt.event.*;

class MouseEventDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Mouse Event");
        JLabel lbl = new JLabel("Move or Click on me");

        lbl.addMouseListener(new MouseAdapter() {
            public void mouseEntered(MouseEvent e) {
                System.out.println("Mouse Entered");
            }
            public void mouseClicked(MouseEvent e) {
                System.out.println("Label Clicked");
            }
        });

        lbl.addMouseMotionListener(new MouseMotionAdapter() {
            public void mouseMoved(MouseEvent e) {
                System.out.println("Mouse moving...
"+e.getX()+" "+e.getY()+"");
            }
        });

        f.add(lbl);
        f.setSize(300,200);
        f.setVisible(true);
    }
}

```

★ 3. Key Events

Triggered when a key is pressed, released, typed.

✓ Example using KeyAdapter

```

import javax.swing.*;
import java.awt.event.*;

class KeyEventDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Key Event");
        JTextField t = new JTextField(20);

        t.addKeyListener(new KeyAdapter() {
            public void keyTyped(KeyEvent e) {
                System.out.println("Typed: " + e.getKeyChar());
            }
        });

        f.add(t);
        f.setSize(300,200);
    }
}

```

```
        f.setVisible(true);
    }
}
```

★ 4. Window Events

Triggered when a window is opened, closing, minimized, activated.

✓ Example using WindowAdapter

```
import java.awt.event.*;
import javax.swing.*;

class WindowEventDemo extends JFrame {
    WindowEventDemo() {
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.out.println("Window Closing");
                System.exit(0);
            }
            public void windowActivated(WindowEvent e) {
                System.out.println("Window Activated");
            }
        });

        setSize(300,200);
        setVisible(true);
    }

    public static void main(String[] args) {
        new WindowEventDemo();
    }
}
```

★ 5. Item Events (Checkbox, Combobox)

Triggered when item is selected/deselected.

✓ Example

```
import javax.swing.*;
import java.awt.event.*;

class ItemEventDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Item Event");
        JCheckBox cb = new JCheckBox("Accept Terms");
```

```

        cb.addItemListener(e -> {
            if(e.getStateChange() == ItemEvent.SELECTED)
                System.out.println("Checked");
            else
                System.out.println("Unchecked");
        });

        f.add(cb);
        f.setSize(300,200);
        f.setVisible(true);
    }
}

```

★ 6. Text Events (TextArea & TextField)

✓ Example

```

import java.awt.event.*;
import javax.swing.*;

class TextEventDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Text Event");
        JTextField tf = new JTextField(20);

        tf.addCaretListener(e -> {
            System.out.println("Text changed: " + tf.getText());
        });

        f.add(tf);
        f.setSize(300,200);
        f.setVisible(true);
    }
}

```

★ 7. Focus Events

Triggered when a component gains or loses focus.

✓ Example

```

import javax.swing.*;
import java.awt.event.*;

class FocusEventDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Focus Event");
    }
}

```

```

        JTextField t1 = new JTextField(20);

        t1.addFocusListener(new FocusAdapter() {
            public void focusGained(FocusEvent e) {
                System.out.println("TextField Focused");
            }
        });

        f.add(t1);
        f.setSize(300,200);
        f.setVisible(true);
    }
}

```

★ 8. Adjustment Event (Scrollbar)

✓ Example

```

import javax.swing.*;
import java.awt.event.*;

class AdjustmentEventDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Adjustment Event");
        JScrollBar sb = new JScrollBar();

        sb.addAdjustmentListener(e -> {
            System.out.println("Scroll value: " + e.getValue());
        });

        f.add(sb);
        f.setSize(300,200);
        f.setVisible(true);
    }
}

```

★ Final Summary — You Now Have

- ✓ All event types
 - ✓ Clear small examples
 - ✓ Adapter class usage
 - ✓ Eclipse steps (drag, drop, rename, attach listeners)
-

If you want, I can also prepare:

- 📄 A ready-to-submit PDF of all these event programs
- 🖼️ A visual diagram of Delegation Event Model
- 📺 A single combined program demonstrating everything in one frame

Just tell me ❤️

You said:

What are all the layouts

ChatGPT said:

Here is the **complete list of all Java AWT/Swing Layout Managers**, grouped, explained simply, and with example diagrams + when to use them.

★ ALL LAYOUT MANAGERS IN JAVA (AWT/SWING)

Layout Managers control *how components are arranged* inside a container (Frame, Panel, JPanel).

1. FlowLayout (Default of JPanel)

Components are placed **left to right**, and wrap to the next line.

✓ When to Use

Simple forms, buttons row, toolbars.

✓ Example

```
[Btn1] [Btn2] [Btn3] [Btn4]
```

Code

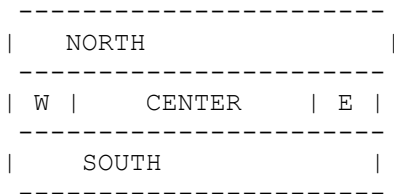
```
setLayout(new FlowLayout());
```

2. BorderLayout (Default of JFrame)

Divides the container into **5 regions**:

- North
- South
- East
- West
- Center

✓ Example



Code

```
setLayout (new BorderLayout ());
```

3. GridLayout

Arranges components in a **fixed rows × columns grid**, all cells equal size.

✓ Example (3×2)

```
[A] [B]  
[C] [D]  
[E] [F]
```

Code

```
setLayout (new GridLayout (3, 2));
```

4. CardLayout

You can stack multiple components and show **one at a time** like cards (wizard, tabs).

✓ Example

```
[Card1]  
[Card2]  
[Card3] ← shown
```

Code

```
setLayout(new CardLayout());
```

5. GridBagLayout (Most Powerful Layout)

Allows fine control over:

- row span
- column span
- alignment
- padding

Used for **complex forms**.

Code

```
setLayout(new GridBagLayout());
```

(Requires GridBagConstraints)

6. BoxLayout

Arranges components **vertically or horizontally**.

✓ Example (Vertical)

```
[Label]  
[TextField]  
[Button]
```

Code

```
setLayout(new BorderLayout(this, BorderLayout.Y_AXIS));
```

Horizontal

```
setLayout(new BorderLayout(this, BorderLayout.X_AXIS));
```

7. GroupLayout

Used by **NetBeans GUI Builder**, excellent for form-like designs.

Pros

Aligns components neatly like HTML tables.

Code (sample)

```
setLayout(new GroupLayout(getContentPane()));
```

8. SpringLayout

Allows components positioned relative to each other with "springs" (flexible spacers).

Code

```
setLayout(new SpringLayout());
```

9. Null Layout (No Layout Manager)

You position components manually using:

```
setBounds(x, y, width, height);
```

Avoid unless necessary

Used in drag-drop apps, games.

★ Summary Table

Layout Manager Default For		Behavior
FlowLayout	JPanel	Left-to-right flow
BorderLayout	JFrame	5 regions
GridLayout	—	Equal grid
CardLayout	—	One panel at a time
GridBagLayout	—	Most flexible (complex forms)
BoxLayout	—	Vertical/Horizontal boxes
GroupLayout	—	Professional form layout
SpringLayout	—	Elastic constraints
Null Layout	—	Manual positioning

If you want, I can make:

-  A PDF with layout diagrams
-  A visual comparison with colors
-  Java programs demonstrating each layout
-  A combined GUI showing all layouts in tabs

Just tell me!

You said:

absolute layout?

ChatGPT said:

Yes! Absolute Layout = Null Layout in Java.

It means:

! There is *no layout manager*.

You manually place every component by specifying:

```
x-coordinate  
y-coordinate  
width  
height
```

This is done using:

```
setLayout(null);  
component.setBounds(x, y, width, height);
```

★ ABSOLUTE LAYOUT (NULL LAYOUT)

- ✓ Components do NOT auto-adjust
 - ✓ You decide exact positions
 - ✓ Works like drawing on a canvas
 - ✓ Common in games, drag-drop apps, small UIs
-

✓ Example: Absolute Layout in Java

```
import javax.swing.*;  
  
class AbsoluteLayoutDemo {  
    public static void main(String[] args) {  
        JFrame f = new JFrame("Absolute Layout");  
        f.setLayout(null);    // ABSOLUTE LAYOUT  
  
        JButton b = new JButton("Click");  
        JTextField t = new JTextField();  
  
        b.setBounds(50, 50, 100, 40);    // x, y, width, height  
        t.setBounds(50, 120, 150, 40);  
  
        f.add(b);  
        f.add(t);  
  
        f.setSize(300,300);  
        f.setVisible(true);  
    }  
}
```

★ Advantages

- ✓ Full control over exact positions
- ✓ Easy for small UIs
- ✓ Perfect for teaching/demo apps

★ Disadvantages

- ✗ Does NOT resize automatically
- ✗ Breaks on different screen sizes / DPI
- ✗ Must manually adjust layout for everything

★ Eclipse WindowBuilder – Using Absolute Layout

To use absolute layout in Eclipse:

1. Open the **Design** tab
2. Right-click the JFrame → **Set Layout** → **Absolute Layout**
3. Drag components → Drop anywhere
4. Resize them freely
5. Eclipse auto-updates `setBounds()` in code

It becomes **WYSIWYG** positioning.
